

CS-GY 9223 Cloud Computing: Social Media Posting Automation

*Swetha Ganesh Babu - sg8554,
Sriram Madhialagan - sm12155,
Morgan Waddington - mw5599,
Divyansh Agarwal - da3245*

<https://github.com/swetha-ganeshbabu/prompt-to-post>

Abstract

This project presents an end-to-end, cloud-native system that automates the creation and posting of captioned social media images using user-provided photos and chatbot input. Built entirely on AWS, the system combines services like Lambda, Cognito, Lex, Rekognition, Bedrock (Claude 3 Haiku), SES, Elasticache, Opensearch, S3, API Gateway, AWS Secrets and SQS to deliver scalable, secure, and personalized content generation. Users can sign in, describe the type of image and caption they want, and receive AI-generated results ready for posting—reducing manual effort and enabling more consistent, engaging social media activity.

Problem Statement

Creating engaging, high-quality social media content is time-consuming and often requires technical skills in design, captioning, and scheduling. Users, especially small creators and businesses, struggle to consistently generate and post content that matches trends and their brand voice. This project aims to solve that by building a fully automated, scalable system that lets users upload or connect their photos, chat with a bot to describe their posting needs, and automatically receive AI-generated captions and overlays—ready to post on platforms like Bluesky—all secured and powered by AWS.

Existing Solutions

Several tools exist today to support content creation and social media posting, but they typically address only parts of the workflow. Platforms like Canva combined with scheduling tools such as Buffer or Later allow users to

design and schedule posts manually. However, these require significant user input and lack AI-powered automation.

Some users rely on language models like ChatGPT or Bard to generate captions, but these approaches are disconnected from image storage and require manual uploads and copy-pasting. Built-in tools such as Meta Business Suite offer basic scheduling and caption suggestions, though they are platform-specific and not personalized.

More experimental setups using agentic systems like Auto-GPT attempt full automation but are often complex to manage, insecure, and not user-friendly. AI-based content platforms like Lately or Jasper AI generate text content based on blogs or websites, but they don't work with personal images or handle end-to-end posting.

In contrast, this project offers a unique solution that fully automates the content generation pipeline—from photo selection and caption creation to visual overlay and social media posting—using a multimodal chatbot interface and a secure, scalable cloud-native backend powered by AWS services.

System Architecture

This project is built on a cloud-native, serverless architecture using Amazon Web Services (AWS). The system is modular, event-driven, and highly scalable to support real-time user interactions and automated content workflows. Each component is designed to fulfill a specific role while working together seamlessly through secure, decoupled integrations. The architecture can be divided into the following stages: data ingestion,

processing and indexing, chatbot interaction, caption generation, image overlay, and automated posting.

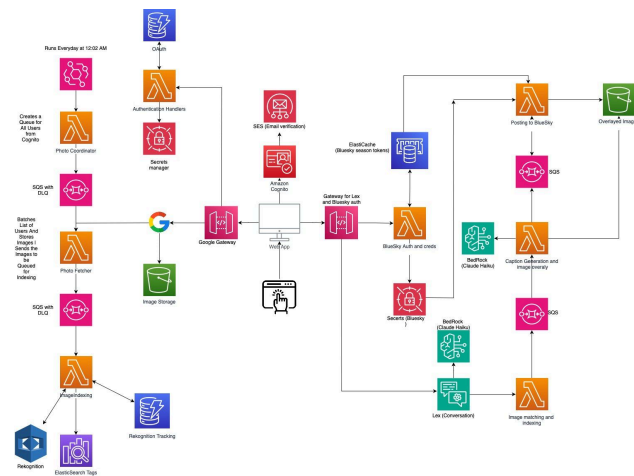


Fig. 1. Final System Architecture of the Prompt-To-Post

User Account Connection and Google Photos Authorization

When a Cognito-authenticated user chooses to connect their Google Photos account, the following flow is triggered:

1. **Initiation:** The frontend sends an authenticated request to an API Gateway endpoint (`/initiate-google-oauth`). An AWS Lambda function validates the user via their Cognito ID token, stores a temporary state in DynamoDB, and retrieves Google OAuth credentials from Secrets Manager.
2. **Redirect to Google:** The Lambda constructs a Google Authorization URL with the state and sends it to the frontend, which redirects the user to Google's consent page.
3. **User Consent and Callback:** After approval, Google redirects the user to a callback endpoint (`/google-oauth-callback`) with an authorization code and the state.
4. **Token Exchange and Storage:** The callback Lambda verifies the state, exchanges the code for access and refresh tokens, and securely stores the refresh token in Secrets Manager under the

user's Cognito ID. The temporary state is removed.

5. **Completion:** The user is redirected back to the frontend. The stored refresh token now enables automated photo fetching during scheduled jobs.

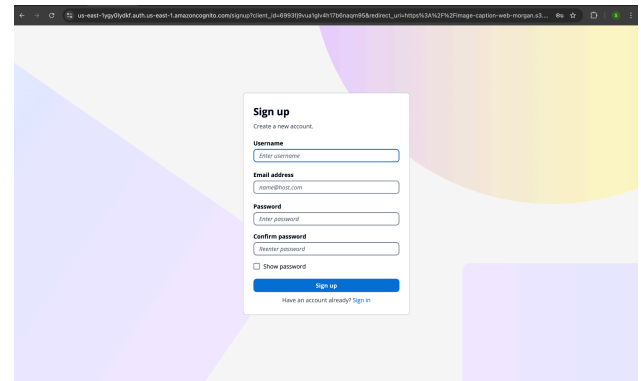


Fig. 2 User sign-up interface for account creation via cognito

Data Ingestion

A scheduled AWS Lambda function, referred to as the **Photo Coordinator**, is triggered daily at 1:02 AM via **Amazon EventBridge** (formerly CloudWatch Events). This function kickstarts the photo ingestion workflow by querying **Amazon Cognito** for all registered users, each uniquely identified by their Cognito sub field.

For every user, a discrete job is constructed and pushed into an **Amazon SQS queue**—a scalable message buffer that decouples processing from ingestion. This queue is configured with a **Dead Letter Queue (DLQ)** to capture failed or malformed jobs, adding robustness to the architecture.

The **Photo Fetcher Lambda** processes each SQS message. For each user, it retrieves OAuth2 tokens securely stored in **AWS Secrets Manager** to connect with the **Google Photos API**. New images added to the user's Google Photos library are fetched and then uploaded to a centralized **Amazon S3 bucket**, chosen for its durability, scalability, and integration with downstream AWS services.

Image Processing and Metadata Indexing

Once images are successfully stored in S3, a downstream Lambda function, **Image Indexing**, is automatically triggered for each new object. This function performs two core tasks:

1. **Image Labeling via Amazon Rekognition:**
Each image is analyzed using **Amazon Rekognition**, which extracts rich metadata including labels, scenes, and object types. This allows for a semantic understanding of image content (e.g., “Dog,” “Beach,” “Sunset”).
2. **Indexing for Query and Search:**
Indexed into **Amazon OpenSearch Service**, enabling full-text search, keyword matching, and content-based image retrieval across all users.

Rekognition Tracking and Data Consistency

To maintain integrity across ingestion, analysis, and searchability, a lightweight tracking mechanism—**Rekognition Tracker**—is used. This ensures that:

- Each uploaded image has been successfully processed by Rekognition,
- Metadata has been written to both DynamoDB and OpenSearch,
- Duplicate or missed images can be identified and reprocessed automatically.

This layered architecture ensures high availability, horizontal scalability, and extensibility for future storage providers or analysis modules.

Authentication and Chatbot Interface

Before user input reaches the captioning backend, it is routed through an API Gateway, which serves as a centralized entry point for external requests. This setup not only standardizes request handling but also enables monitoring, throttling, and authorization control. When a user makes a request such as “post a morning coffee,” the API Gateway validates and forwards the input to the

corresponding Lambda function responsible for querying the image database

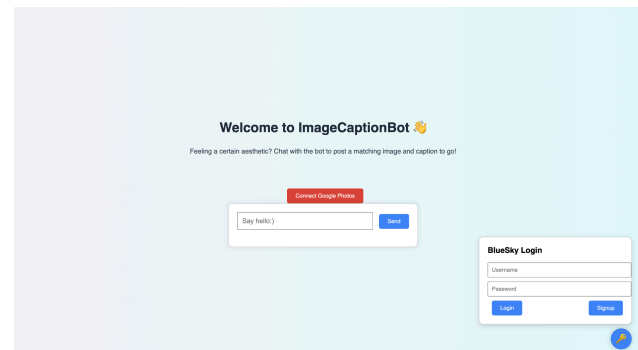


Fig. 3 Combined interface for chatbot input, photo access, and BlueSky login.

User interaction begins at a static frontend hosted on Amazon S3. The interface is a simple chatbot that guides users through the process of generating and posting content. Authentication is managed using Amazon Cognito's Hosted UI, which supports secure sign-in via OAuth 2.0. Upon login, users receive an ID token from the Cognito User Pool. This token is then exchanged for temporary AWS credentials through a Cognito Identity Pool, which provides the frontend with limited and secure access to services like Amazon Lex and S3.

This approach offloads security to AWS, reduces the attack surface, and allows fine-grained IAM controls. It also supports federated identities, which makes it easy to integrate other login providers like Google in the future.

The chatbot is powered by Amazon Lex V2. Users can describe the image they want to post and specify a caption style in natural language. Lex parses the input into structured slots and forwards the request to a backend Lambda function for processing.

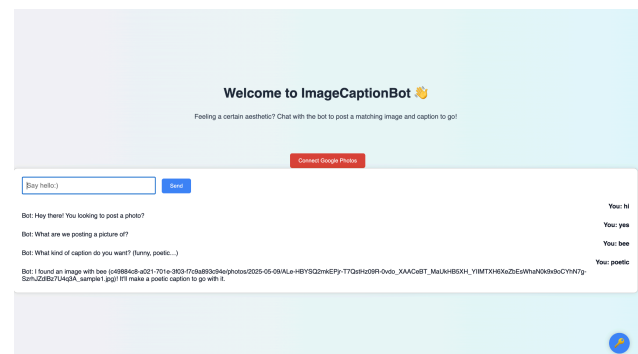


Fig. 4 User input captured and matched with an image using Lex.

To facilitate structured input from users, the chatbot is configured with multiple intents. The core intent, `RetrieveImageIntent`, is designed to capture the user's request to post an image along with their preferred caption tone. This intent extracts two key slots—`imageTopic` and `captionStyle`—both of which use free-form input to allow maximum natural language flexibility. Sample utterances like “I want to post a photo” or “Show me a picture” help Lex recognize user intent even with varied phrasing. Additional intents such as `GreetingIntent` and `FallbackIntent` improve the overall interaction quality by handling casual inputs and providing fallback responses when user messages don't match known patterns. This multi-intent setup allows the chatbot to remain conversational while capturing precise parameters needed for downstream AI processing.

Backend Lambda: Matching and Dispatching Tasks

The Lambda function triggered by Lex is the core of the captioning pipeline. It first uses Claude 3 Haiku via Amazon Bedrock to normalize and clean the user's input. The function constructs a well-defined prompt and sends it to Claude, which returns a structured JSON containing the topic and style fields. This eliminates ambiguities and ensures consistent downstream behavior.

Next, the Lambda fetches the user's unique sub from Cognito using the `admin_get_user` API. This sub acts as the user identifier in OpenSearch queries. The function then searches for a matching image by filtering for the specified topic and user ID. If a suitable image is found, the Lambda constructs a task payload and places it in the `ImageCaptioning` SQS queue. This queue allows captioning and overlay processing to be handled asynchronously, improving performance and reliability.

This decoupling also helps avoid timeouts and cold starts in synchronous workflows. We initially experimented with a fully chained Lambda pipeline, but it proved fragile and unscalable under high traffic. Using SQS introduced much-needed modularity and fault tolerance.

Model Selection and Integration for Caption Generation

To generate personalized, high-quality captions, we evaluated a wide range of AI models and strategies capable of handling natural language and visual input. Our goal was to develop a robust captioning pipeline that could translate free-form user intent into stylized captions, suitable for both image overlay and social media posting.

To improve reliability and flexibility, we experimented with several open-source AI models, starting with Hugging Face's BLIP. BLIP performed reasonably well on local tests, generating basic captions from images, but it lacked nuanced control over tone and output structure.

To address these shortcomings, we explored BLIP-2, which provided better results but required GPU acceleration, making it impractical for serverless deployment. Testing continued in Google Colab, where we encountered significant latency and memory constraints. We also tested Google's Flan-T5 for text style control; however, its lack of native image understanding and inconsistent integration with visual input limited its usefulness. Attempts to combine BLIP for visual features and Flan-T5 for caption refinement yielded inconsistent and brittle outputs.

Based on these limitations, we sought models better suited for multimodal inputs. After consulting with our mentor, we turned to Amazon Bedrock's suite of foundation models. Amazon Titan and Nova were initially evaluated, but Titan lacked image support and Nova did not return structured, parse-friendly output. Ultimately, we selected Claude 3 Haiku, a multimodal model offered through Bedrock, which supported both base64-encoded image input and highly customizable prompt engineering.

Claude 3 Haiku became the cornerstone of our caption generation pipeline. It provided fast, structured responses compatible with AWS Lambda execution, and returned output in a clean, predictable format. The JSON included a short caption (under 10 words) suitable for overlay, a longer caption with hashtags for posting, and embedded styling based on user preferences. This structure minimized downstream logic and enabled scalable, reliable automation.

The Lambda function that invokes Claude is triggered by messages in the `ImageCaptioning` SQS queue. Each message includes the user's Cognito sub (ID), image S3 path, and preferred caption style. The function fetches the

image, encodes it in base64, builds a prompt combining user style and extracted labels, and calls Bedrock’s Claude runtime. The response is parsed to extract both captions and used to generate up to ten hashtags using regex filtering.

For image preprocessing, we needed the Pillow library for caption rendering. While AWS Lambda does not include Pillow by default, we initially explored compiling it into a custom Docker-based layer. To simplify deployment, we ultimately integrated the pre-built Klayers-p312-pillow Lambda Layer for Python 3.12, which provided full compatibility with our overlay requirements. This model engineering journey—from open-source exploration to a scalable Bedrock-powered Lambda function—resulted in a robust, decoupled system capable of producing expressive, consistent captions in real time. Claude 3 Haiku’s structured output played a central role in reducing complexity and ensuring that caption generation integrated seamlessly with downstream image overlay and posting components.

Image Overlay and Visual Output

Once a short-caption message is received from the ImageCaptioningSQS, the Overlay Worker Lambda immediately downloads the raw image from S3 and parses out both the concise overlay text and the accompanying long caption (with hashtags and user ID) provided by Bedrock (Claude Haiku). The image bytes are decoded into a Pillow Image object, at which point the function dynamically selects an appropriate TrueType font—sourced from a custom Lambda Layer built atop Amazon Linux—and calculates optimal font size, line breaks, alignment, and a subtle drop-shadow effect to ensure legibility against varying backgrounds. After rendering the overlay text directly onto the image, the worker re-encodes the modified asset and writes it to the “posting-photos” S3 bucket under a timestamped key, preserving both the original and each successive version for auditability and rollback.

Prior to any social posting, users must authenticate their Bluesky accounts via the web application. At login, the frontend submits the user’s Bluesky username and password to the BlueskyAuth Lambda through API Gateway. That function invokes Bluesky’s `com.atproto.server.createSession` endpoint, receives an `accessJwt`, and then securely stores the raw credentials in

AWS Secrets Manager while caching the session token in ElastiCache (with a TTL matching the token’s expiry). This setup guarantees that downstream Lambdas never embed credentials in code and always access a fresh, short-lived token without incurring latency for repeated logins.

With the overlay complete and tokens in place, the Overlay Worker constructs a single JSON payload—containing the S3 URI of the overlaid image, the long caption text, user ID, and hashtags—and publishes it to the Posting_queue SQS. By decoupling image processing from social-media interactions, the system ensures that transient failures in the posting step cannot back-pressure or delay the higher-volume overlay pipeline.

The Posting_to_social_media Lambda, triggered by messages on posting_queue, orchestrates the final API flow. It first looks up the user’s `accessJwt` in ElastiCache; if the token is missing or expired, it transparently fetches the raw credentials from Secrets Manager and re-authenticates to obtain a new JWT. Next, the function uploads the overlaid image via Bluesky’s media endpoint (`uploadBlob`), which returns a `blobRef` pointer. Finally, it calls the feed-post API (`app.bsky.feed.post`), supplying the `blobRef` along with the long caption and hashtags. Success responses are logged and surfaced to our monitoring dashboard; failures are routed to a dead-letter queue for automated retry.

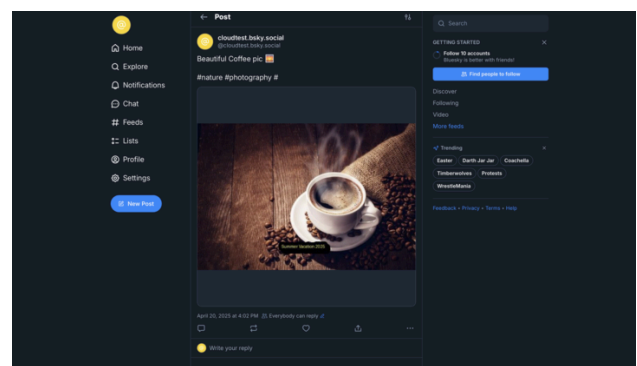


Fig. 5. Screenshot of the final overlaid image and caption as posted on Bluesky

By isolating overlay rendering, authentication, and posting into separate, SQS-chained Lambdas—with Pillow packaged as a self-contained layer and credentials managed via Secrets Manager + ElastiCache—this design achieves high concurrency, fault isolation, and secure,

low-latency access to user tokens. The result is a robust, scalable component that cleanly bridges AI-generated captions to polished, shareable images in Bluesky.

System Properties: Scalability, Security, and Availability

Scalability is achieved through the serverless nature of AWS Lambda, which automatically provisions instances to meet incoming demand. Amazon SQS helps buffer and queue heavy workloads, preventing system overload. Both Lex and Bedrock are managed services that scale horizontally.

Security is enforced through Cognito’s authentication flow, which includes secure token handling and user-specific IAM policies. Secrets Manager protects sensitive information such as Google OAuth tokens and Bluesky credentials. All inter-service communication is authenticated, and no credentials are hardcoded.

Availability is guaranteed through the use of fully managed AWS services like S3, DynamoDB, OpenSearch and ElastiCache. These services offer built-in redundancy and failover capabilities, ensuring that the system remains online and responsive even under failure conditions.

This layered, loosely coupled architecture provides the flexibility to grow, the security to protect users, and the robustness to support real-world deployment scenarios. Every component was chosen after evaluating multiple options, prioritizing reliability, extensibility, and developer maintainability.

Explanation of the choices made

Much of this paper already details the specific technologies used and the reasons behind their selection. To briefly summarize, our architectural choices were guided by the need for scalability, modularity, and ease of maintenance. We prioritized managed, serverless services to reduce operational burden and to allow different parts of the system to evolve independently. AWS Lambda, SQS, and Bedrock provided a highly decoupled pipeline that could scale with demand and support AI integration without requiring custom infrastructure. Cognito and Secrets Manager ensured secure identity and credential management with minimal friction. In areas where initial solutions proved limiting—such as manual input parsing

or tightly coupled workflows—we iterated quickly, replacing them with more cloud-native and resilient components. Overall, the system reflects a balance between rapid prototyping and long-term viability.

Future Work

While the current system successfully integrates AWS services to automate image-based caption generation and posting, several enhancements are planned to further improve performance, user experience, and security.

One potential improvement involves refining how user identity is handled across services. At present, the Lambda backend queries Cognito to retrieve each user’s unique ID (sub). In future iterations, this identifier could be passed directly from the frontend as part of the session attributes, eliminating the need for an extra API call and reducing latency in the chatbot response loop.

Security can also be strengthened by migrating currently hardcoded OpenSearch credentials into AWS Secrets Manager. While the system already uses Secrets Manager for sensitive third-party tokens, centralizing all credential storage would ensure tighter control and align fully with AWS security best practices.

Another area of future exploration is resilience at scale. As more users interact with the system simultaneously, implementing retry logic with exponential backoff around key services like Amazon Bedrock and OpenSearch would help gracefully handle rate limits and transient network issues. Although these services are already highly scalable, incorporating more robust retry mechanisms will improve reliability under peak usage.

From a frontend perspective, user interaction can be enriched with dynamic feedback—such as real-time status indicators while images are being matched or captions are being generated. These additions would create a more polished and intuitive user experience.

Finally, the system is well-positioned for broader integration. With foundational support for user authentication and OAuth already in place, connecting to additional photo storage services or expanding posting options beyond Bluesky could be accomplished with minimal architectural changes.

These improvements are not corrections but natural next steps in evolving a working system into a production-grade platform that scales smoothly, operates securely, and delivers a delightful experience to end users.

Results

Throughout development, the system was tested end-to-end with real user interactions, photo uploads, and captioning requests. Users were able to authenticate via Cognito, connect their Google Photos account, authenticate with Bluesky and converse with the Lex-powered chatbot to describe the type of image and caption they wanted. The system successfully retrieved matching images using OpenSearch filters scoped to the user's ID and selected labels.

Claude 3 Haiku, invoked through Amazon Bedrock, consistently returned structured and relevant topic/style pairs even from loosely phrased or abstract user inputs. Generated captions varied in tone and length as requested, and the short captions were successfully overlaid on the correct image using Pillow, resulting in a visually clean, stylized output.

Each step of the pipeline—from ingestion to caption generation, image overlay and posting—was validated in isolation and then in combination. The system remained stable under concurrent requests, with SQS buffering loads and Lambda scaling as needed. Logging and monitoring via CloudWatch provided visibility into failures and confirmed the effectiveness of the Dead Letter Queue in isolating problematic inputs.

While full social media automation is currently limited to Bluesky due to API cost constraints with platforms like Twitter/X, the final captioned images and hashtags were correctly posted via the Bluesky API. Credentials were managed securely per user using Secrets Manager, and token refresh via Lambda was successfully triggered both on schedule and on-demand.

Overall, the system met its goal of reducing the manual effort required to create and post visually engaging, captioned content. It demonstrates a scalable, AI-assisted approach to creative automation and serves as a strong foundation for future expansion and refinement.